



Alma Mater Studiorum-Università di Bologna Scuola di Ingegneria

Java: wrapper per tipi primitivi

Corso di Laurea in Ingegneria Informatica
Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

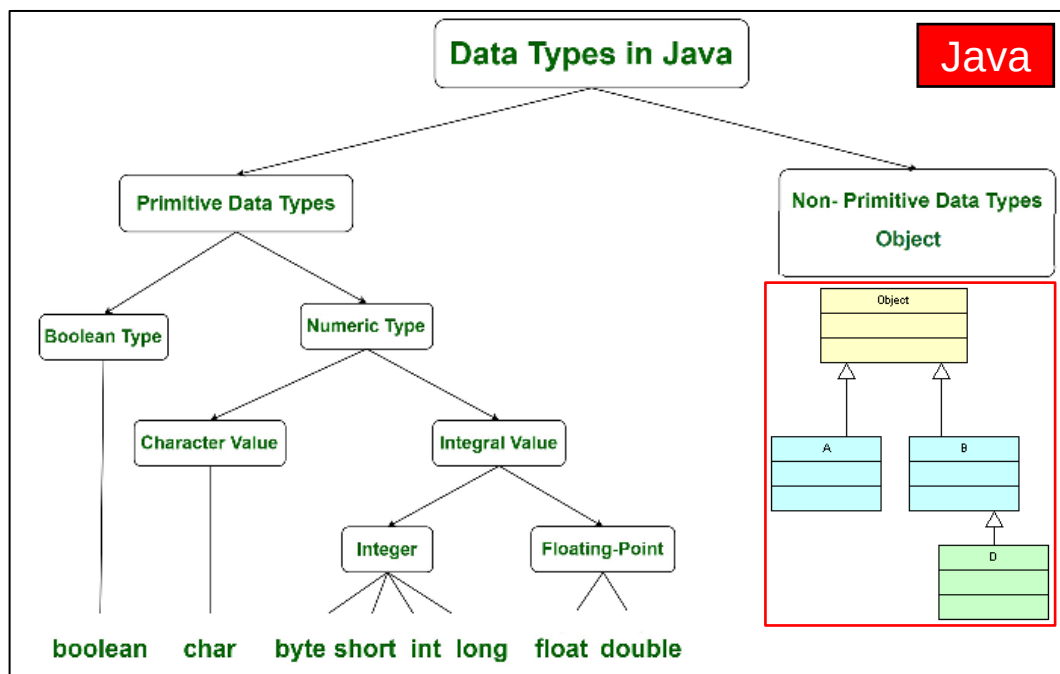


UN PROBLEMA: I TIPI PRIMITIVI

- A differenza di C#, Scala e Kotlin, **in Java i tipi base *non sono classi***: richiedono sempre un trattamento ad hoc
- A causa di tale disuniformità:
 - non hanno metodi: le operazioni sono funzioni statiche di libreria
 - usano `==` invece di `equals`
 - adottano il passaggio per valore anziché per riferimento
 - hanno tipi-array separati
 - ***non supportano ereditarietà e polimorfismo***
 - più in generale, ***non si possono usare dove è richiesto un oggetto***
 - nei tipi varianti
 - nelle collection
 - negli stream di operazioni
 - ...

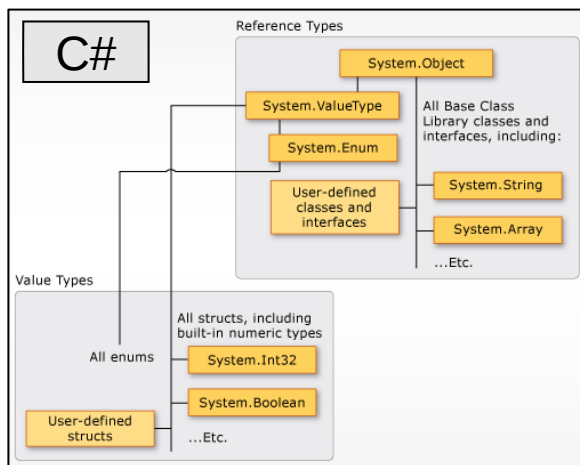
JAVA: TASSONOMIA DI TIPI

- In Java i *tipi base non sono classi*: appartengono a un ramo separato della tassonomia dei tipi
 - **Object** è il tipo-base capostipite di tutti i *tipi riferimento*
 - MA i tipi primitivi stanno altrove
- Perciò, i *valori di tali tipi non sono oggetti*
 - non si può usare un valore primitivo dove ci si attenda un **Object**

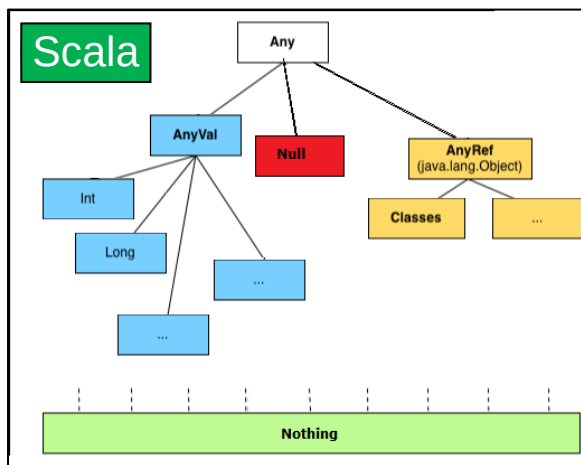


NEGLI ALTRI LINGUAGGI, INVECE...

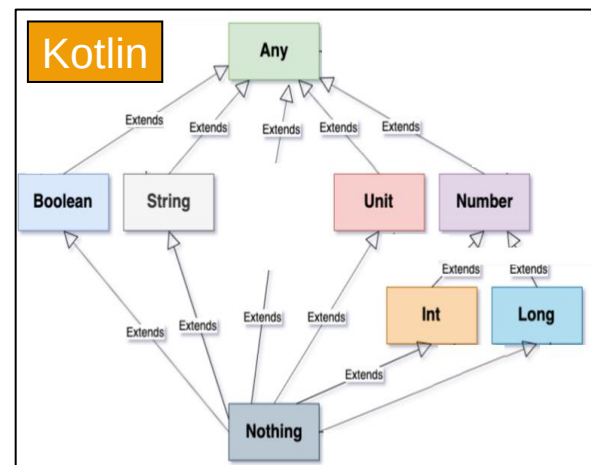
- Non è così in C#, Scala, Kotlin
 - con soluzioni diverse, in tutti quei linguaggi i tipi base *sono* classi e i relativi valori *sono* oggetti → uniformità ☺



In C#, opportuni alias simulano la presenza dei tipi base: ad esempio,
`System.Int32` → `int`



In Scala, la tassonomia prevede un ramo con *tipi valore* **`Int`, `Long`, `Float`**, etc. che derivano da **`AnyVal`**



In Kotlin, la tassonomia prevede *tipi valore* **`Int`, `Long`, `Float`**, come sottoclassi dirette (final) di **`Any`**



TIPI PRIMITIVI IN JAVA: PROBLEMI

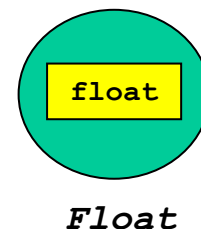
- Poiché i tipi base Java sono tipi primitivi, *non classi*, non si può usare un valore primitivo dove sia atteso un **Object**
 - ad esempio, in un array di `Object` non si *potrebbero* inserire valori primitivi:

Java

```
Object[] array = new Object[10];  
array[0] = "ciao";           // OK, String è un Object  
array[1] = new Counter(2);   // OK, Counter è un Object  
array[2] = new Frazione(3,4); // OK, Frazione è un Object  
  
array[3] = 7;                // non si potrebbe assegnare un int a un Object  
array[4] = 7.2;              // non si potrebbe assegnare un double a un Object
```

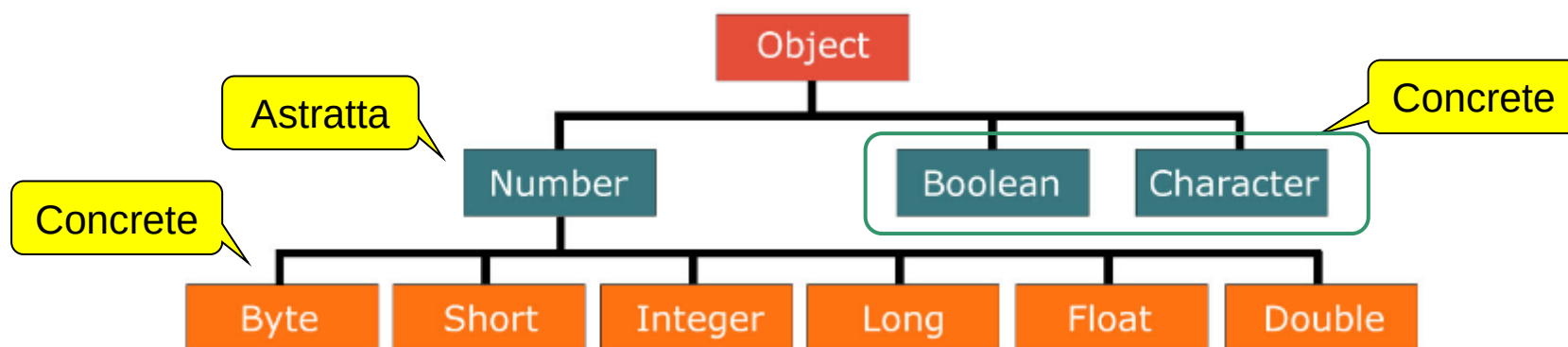
TIPI PRIMITIVI: SOLUZIONE

- La soluzione Java per garantire interoperabilità si basa sulla definizione di opportune *classi wrapper*
- Ogni *wrapper class* incapsula un valore di tipo primitivo
- Di base, la *wrapper class* è «quasi omonima» al corrispondente tipo primitivo...
 - classe **Long** per incapsulare il tipo primitivo **long**
 - classe **Float** per incapsulare il tipo primitivo **float**
 - classe **Double** per incapsulare il tipo primitivo **double**
 - ...etc.
- .. ma ci sono alcune eccezioni.



CLASSI WRAPPER IN JAVA

- Ogni tipo primitivo ha una **classe wrapper** «*quasi omonima*»
 - di norma, stesso nome con iniziale maiuscola
 - tranne i due casi `int` → `Integer`, `char` → `Character`



- Le classi wrapper **numeriche** derivano dalla **classe astratta Number**, che definisce svariati metodi di conversione
 - `intValue`, `longValue`, `floatValue`, `doubleValue`, etc.



CLASSI WRAPPER IN JAVA: BOXING & UNBOXING

- L'operazione di **incapsulamento di un valore primitivo** nel corrispondente **oggetto wrapper** si chiama **BOXING**
 - fino a Java 8, si faceva tramite il costruttore del wrapper

```
Integer i = new Integer(22);  
Double d = new Double(3.14);
```

Java
 - da Java 9 tale approccio è deprecato: il costruttore pubblico sarà presto *rimosso*, nella direzione di avere vere *value-based classes*
 - nuovo approccio: **costruzione tramite factory → `valueOf`**

```
Integer i = Integer.valueOf(22);  
Double d = Double.valueOf(3.14);
```

Java

 - ottimizza la creazione delle istanze, mediante caching
 - ***value-based classes***: «object identity should not matter»



VERSO VALUE-BASED CLASSES

- Il concetto di *value-based class* è già sfruttato in Java per `Optional` e le classi del package `java.time`
- Le istanze di una *value-based class*:
 - sono *final* e *immutabili* (possono però referenziare oggetti mutevoli)
 - utilizzano solo `equals` per l'uguaglianza: non usano mai operazioni sensibili all'identità (come `==` o `hashCode`)
 - non espongono costruttori: sono istanziate solo da metodi factory, per non dare/fare ipotesi sulla *identità* degli oggetti restituiti
 - `equals`, `hashCode`, `toString` sono basate solo sullo *stato* dell'istanza stessa (non sulla sua identità o sullo stato di altri oggetti)
 - perciò, due istanze uguali secondo `equals` *sono sempre liberamente intercambiabili* senza che ciò causi modifiche di comportamento



VERSO VALUE-BASED CLASSES

- Con i vecchi costruttori espliciti, venivano create istanze distinte anche per lo stesso valore:

```
jshell> Integer i = new Integer(18)
i ==> 18

jshell> Integer j = new Integer(18)
j ==> 18

jshell> i==j
$3 ==> false
```

Java

- Con i nuovi factory method, ciò non accade:

```
jshell> Integer u = Integer.valueOf(17)
u ==> 17

jshell> Integer v = Integer.valueOf(17)
v ==> 17

jshell> u==v
$6 ==> true
```

Java



CLASSI WRAPPER IN JAVA: BOXING & UNBOXING

- Analogamente, l'estrazione del valore primitivo dal wrapper si chiama **UNBOXING** ed è svolta dai metodi **xxxValue**

Modifier and Type	Method	Description	Java
byte	<code>byteValue()</code>	Returns the value of the specified number as a byte.	
abstract double	<code>doubleValue()</code>	Returns the value of the specified number as a double.	
abstract float	<code>floatValue()</code>	Returns the value of the specified number as a float.	
abstract int	<code>intValue()</code>	Returns the value of the specified number as an int.	
abstract long	<code>longValue()</code>	Returns the value of the specified number as a long.	
short	<code>shortValue()</code>	Returns the value of the specified number as a short.	

- di norma, si usa il metodo di estrazione «corrispondente» al tipo:

```
int x      = i.intValue() ;           // x vale 22  
double z = d.doubleValue() ;        // z vale 3.14
```

- tuttavia, sono sempre disponibili tutti i metodi, che restituiscono il valore *convertito* nel tipo richiesto



CLASSI WRAPPER IN JAVA: BOXING & UNBOXING

- Esempi:

Java

```
Integer i = Integer.valueOf(22);
System.out.println(i.intValue());           // 22
System.out.println(i.doubleValue());        // 22.0

Double d = Double.valueOf(3.14);
System.out.println(d.doubleValue());        // 3.14
System.out.println(d.intValue());           // 3

Float f = Float.valueOf(1.73F);
System.out.println(f.floatValue());         // 1.73
System.out.println(f.doubleValue());        // 1.730000019...
System.out.println(f.longValue());          // 1
```



BOXING & UNBOXING AUTOMATICI

- Tuttavia, l'uso esplicito di boxing & unboxing renderebbe il codice molto (inutilmente) prolisso:

- sia nelle operazioni

```
Integer k = Integer.valueOf( i.intValue()+6 );
```

- sia nell'uso con le collection

```
myList.add( Integer.valueOf(7) );
```

Java

- Per questo, da Java 5 **boxing e unboxing sono automatici**
 - gli automatismi garantiscono l'approccio *value-based*

```
jshell> Integer k = 16
k ==> 16

jshell> Integer h = 16
h ==> 16

jshell> k==h
$9 ==> true
```

```
jshell> int z = k
z ==> 16

jshell> int x = k-1+3*h
x ==> 63
```



TIPI PRIMITIVI & ARRAY

- Grazie ai tipi wrapper e al boxing automatico, l'esempio iniziale in realtà è accettabile e funziona
 - i valori primitivi 7 (`int`) e 7.2 (`double`) vengono *automaticamente convertiti in due oggetti, rispettivamente di tipo `Integer` e `Double`*, che come tali possono essere inseriti in un array di `Object`:

```
Object[] array = new Object[10];  
array[0] = "ciao";           // OK, String è un Object  
array[1] = new Counter(2);   // OK, Counter è un Object  
array[2] = new Frazione(3,4); // OK, Frazione è un Object  
  
array[3] = 7;           // accettato perché il valore int è convertito in Integer  
array[4] = 7.2;        // accettato perché il valore double è convertito in Double
```

Java



TIPI PRIMITIVI & COLLECTION

- Viceversa, non derivando da `Object`, i tipi primitivi *non si possono usare per tipizzare le strutture dati «collection»*, come `List`, `Set`, etc., che lavorano *solo* con oggetti
 - non si può scrivere `List<int>`, `Set<long>`, etc.
 - si dovrà invece scrivere `List<Integer>`, `Set<Long>`, etc.
 - inserire/togliere valori primitivi in tali strutture dati sarà poi possibile, ma al prezzo di un continuo boxing/unboxing (seppur automatici)

```
java.util.List<Integer> listOfInt = new java.util.ArrayList<>();
listOfInt.add(23); // boxing: int --> Integer
listOfInt.add(24); // boxing: int --> Integer
listOfInt.add(25); // boxing: int --> Integer
int x = listOfInt.get(0); // boxing: int --> Integer
int y = listOfInt.get(1); // boxing: int --> Integer
int z = listOfInt.get(2); // boxing: int --> Integer
System.out.println(x);
System.out.println(y);
System.out.println(z);
```

Java

23
24
25

NEGLI ALTRI LINGUAGGI, INVECE...

- In C#, Scala, Kotlin il problema invece non si pone: i tipi primitivi *sono* classi e i relativi valori *sono* oggetti
 - quindi sono usabili in collection, array, etc. senza boxing (almeno in apparenza: a livello implementativo, poi... ☺)

```
var list = new System.Collections.Generic.List<int>();  
list.Add(23);  
list.Add(24);  
list.Add(25);  
foreach(int i in list) Console.WriteLine(i);
```

In C#, **int** è una vera classe
(alias per **System.Int32**)

C#

```
val list = scala.collection.mutable.ListBuffer[Int]();  
list += 23;  
list += 24;  
list += 25;  
for (i ← list) println(i)
```

Int è una
vera classe

Scala

```
val list = kotlin.collections.ArrayList<Int>();  
list.add(23);  
list.add(24);  
list.add(25);  
for (i in list) println(i)
```

Int è una
vera classe

Kotlin